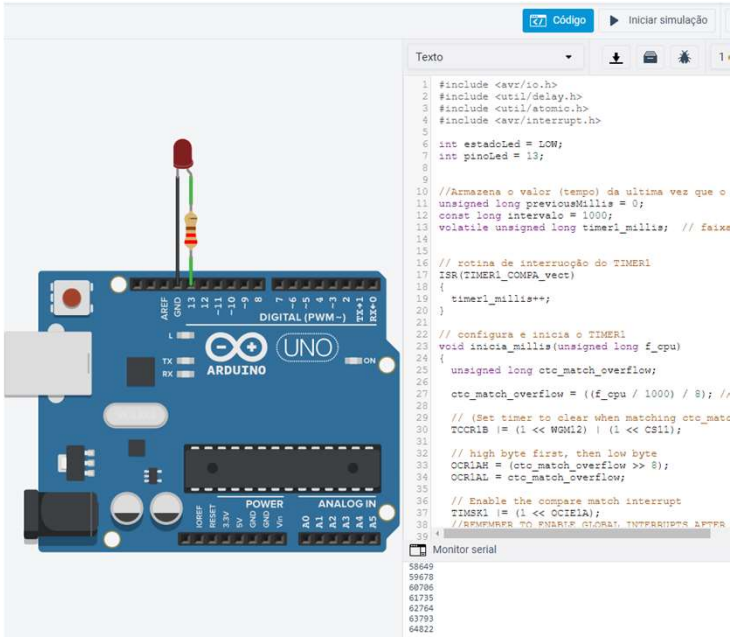
Fundamentos Sistemas Embarcados

Timer ATMEGA328 e função millis()



Fundamentos Sistemas Embarcados

Contador TIMER1

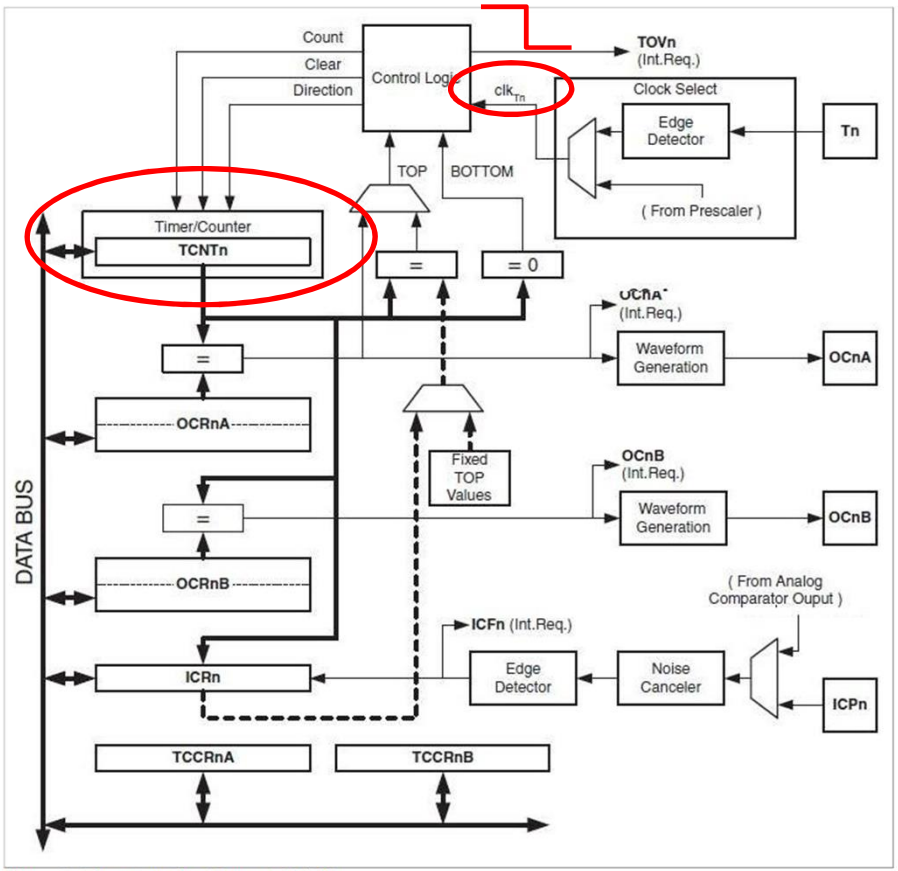


Figure 2: Timer1 on the ATmega168/328

Fundamentos Sistemas Embarcados

Interrupção

```
// rotina de interruoção do TIMER1
ISR(TIMER1_COMPA_vect)
{
  timer1_millis++;
}
```

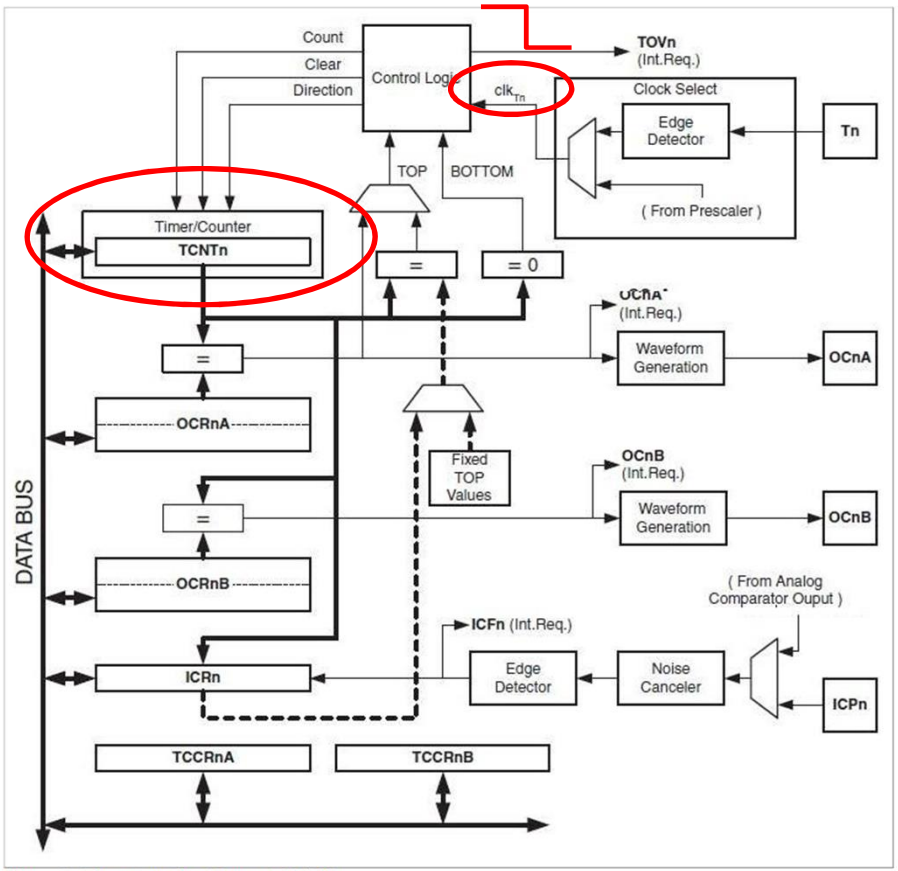


Figure 2: Timer1 on the ATmega168/328

Fundamentos Sistemas Embarcados

Rotina de Interrupção

```
// rotina de interruoção do TIMER1
ISR(TIMER1_COMPA_vect)
{
  timer1_millis++;
}


int main (void)
{
  inicia_millis(16000000UL);
  pinMode(pinoLed, OUTPUT);
  Serial.begin(9600);
  while (1)
 {
    unsigned long currentMillis = nossamillis();
    Serial.println(currentMillis);
     _delay_ms(1000);
 }
}
```

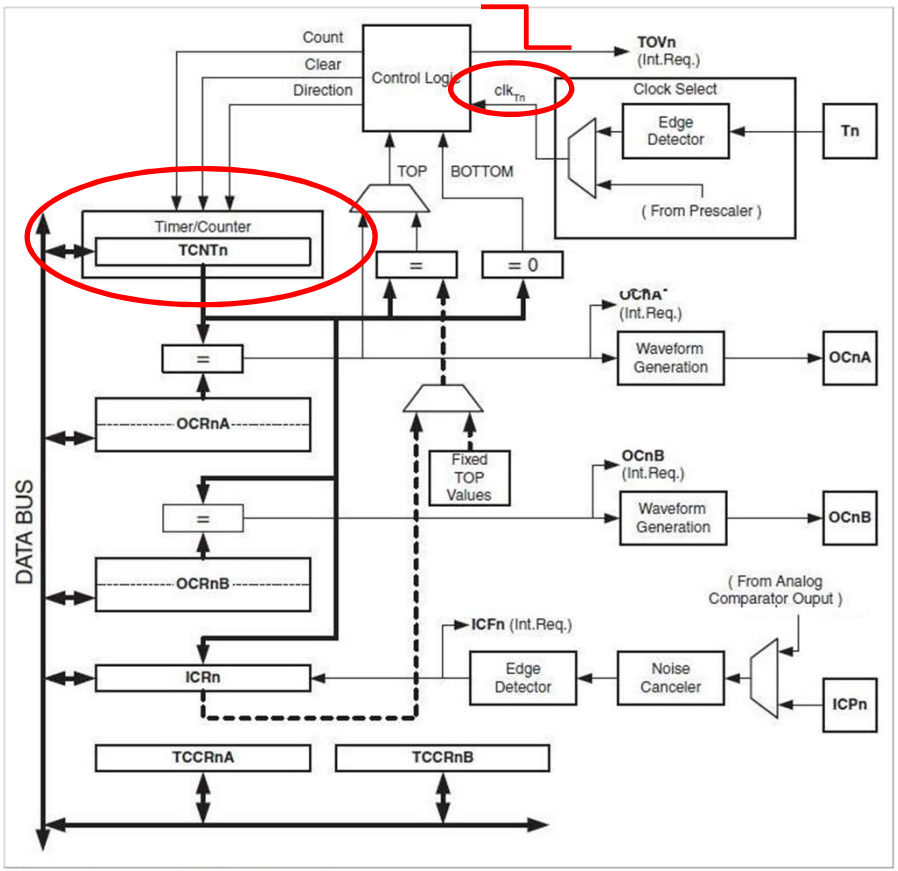


Figure 2: Timer1 on the ATmega168/328

Fundamentos Sistemas Embarcados

Rotina de Interrupção

```
// rotina de interruoção do TIMER1
ISR(TIMER1_COMPA_vect)
{
  timer1_millis++;
}

unsigned long nossamillis ()
{
  unsigned long millis_return;
  // bloca execucao
  ATOMIC_BLOCK(ATOMIC_FORCEON) {
  millis_return = timer1_millis;
  }
  return millis_return;
}


unsigned long currentMillis = nossamillis();
```

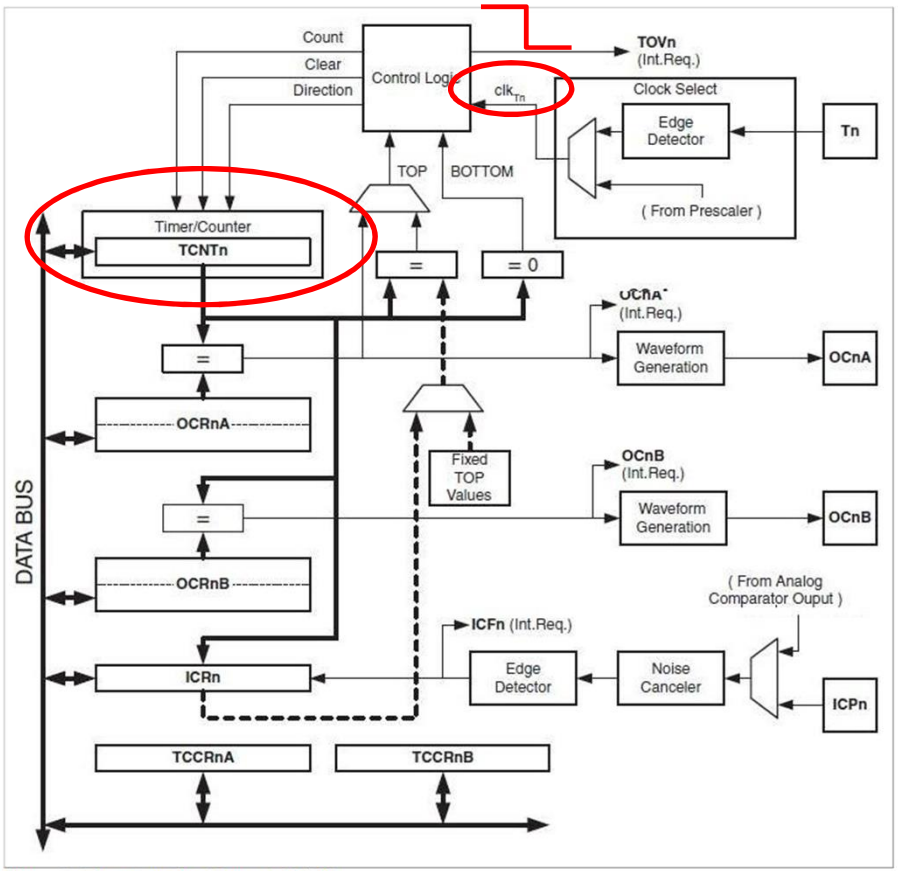


Figure 2: Timer1 on the ATmega168/328

Rotina de Interrupção

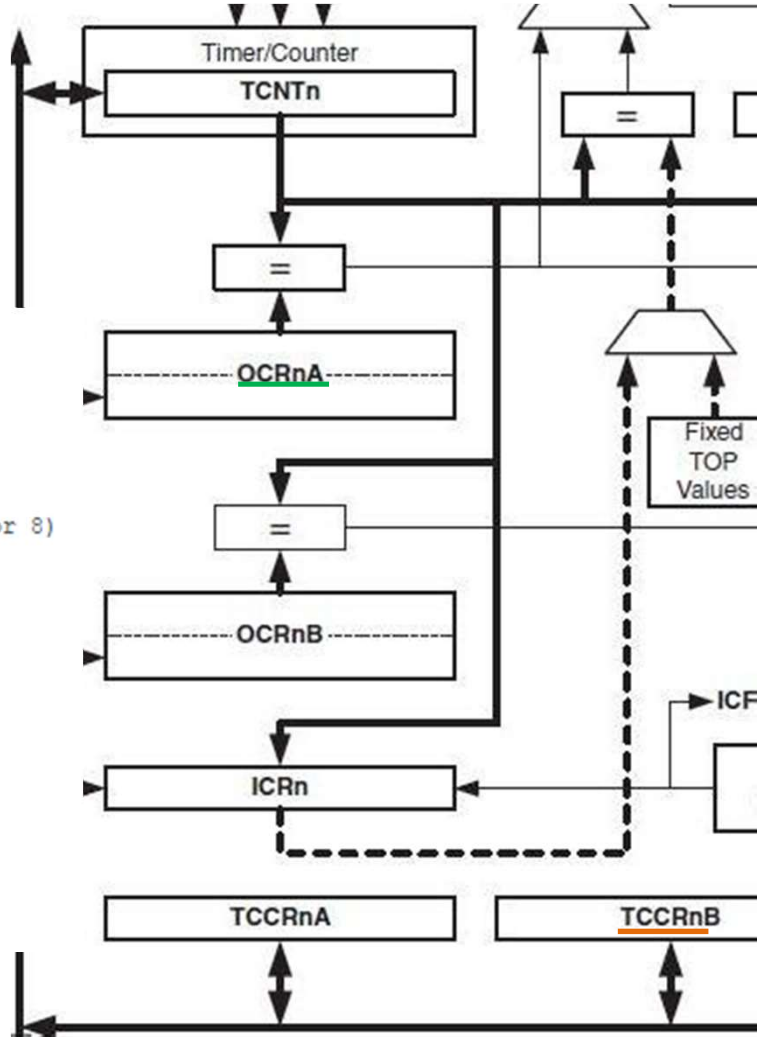
```
// configura e inicia o TIMER1
void inicia_millis(unsigned long f_cpu)
{
  unsigned long ctc_match_overflow;
  ctc_match_overflow = ((f_cpu / 1000) / 8); // overflow do timer em 1ms

  // (limpar timer qdo cooresponder a ctc_match_overflow) | (clock divisor por 8)
  TCCR1B |= (1 << WGM12) | (1 << CS11);

  // high byte first, then low byte
  OCR1AH = (ctc_match_overflow >> 8);
  OCR1AL = ctc_match_overflow;

  // Enable the compare match interrupt
  TIMSK1 |= (1 << OCIE1A);

  //habilita GLOBAL INTERRUPTS
  sei();
}
```



Fundamentos Sistemas Embarcados

Rotina de Interrupção

```
unsigned long ctc_match_overflow;
ctc_match_overflow = ((f_cpu / 1000) / 8);


 // high byte first, then low byte
 OCR1AH = (ctc_match_overflow >> 8);
 OCR1AL = ctc_match_overflow;
```

	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
OCR1AH								
OCR1AL								

Output Compare Register A (stores the compare value, 16 bit)

Fundamentos Sistemas Digitais

Rotina de Interrupção

```
unsigned long ctc_match_overflow;
ctc_match_overflow = ((f_cpu / 1000) / 8);

 // high byte first, then low byte
 OCR1AH = (ctc_match_overflow >> 8);
 OCR1AL = ctc_match_overflow;
```

	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
OCR1AH								
OCR1AL								

Output Compare Register A (stores the compare value, 16 bit)

Fundamentos Sistemas Embarcados

```
// (limpar timer qdo cooresponder a ctc_match_overflow) | (clock divisor por 8)
TCCR1B |= (1 << WGM12) | (1 << CS11);
```

	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
TCCR1B	ICNC1	ICES1	-	WGM13	WGM12	CS12	CS11	CS10

Timer/Counter Control Register 1 B

CS12	CS11	CS10	DESCRIPTION
0	0	0	Timer/Counter1 Disabled
0	0	1	No Prescaling
0	1	0	Clock / 8
0	1	1	Clock / 64
1	0	0	Clock / 256

Fundamentos Sistemas Embarcados

```
// (limpar timer qdo cooresponder a ctc_match_overflow) | (clock divisor por 8)
TCCR1B |= (1 << WGM12) | (1 << CS11);
```

	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
TCCR1B	ICNC1	ICES1	-	WGM13	WGM12	CS12	CS11	CS10

Timer/Counter Control Register 1 B

MODE	WGM13	WGM12	WGM11	WGM10	DESCRIPTION	TOP
0	0	0	0	0	Normal	0xFFFF
1	0	0	0	1	PWM, Phase Corrected, 8bit	0x00FF
2	0	0	1	0	PWM, Phase Corrected, 9bit	0x01FF
3	0	0	1	1	PWM, Phase Corrected, 10bit	0x03FF
4	0	1	0	0	CTC	OCR1A

Fundamentos Sistemas Embarcados

```
// (limpar timer qdo cooresponder a ctc_match_overflow) | (clock divisor por 8)
TCCR1B |= (1 << WGM12) | (1 << CS11);
```

	7 bit	6 bit	5 bit	4 bit	3 bit	2 bit	1 bit	0 bit
TCCR1B	ICNC1	ICES1	-	WGM13	WGM12	CS12	CS11	CS10

Timer/Counter Control Register 1 B

MODE	WGM13	WGM12	WGM11	WGM10	DESCRIPTION	TOP
0	0	0	0	0	Normal	0xFFFF
1	0	0	0	1	PWM, Phase Corrected, 8bit	0x00FF
2	0	0	1	0	PWM, Phase Corrected, 9bit	0x01FF
3	0	0	1	1	PWM, Phase Corrected, 10bit	0x03FF
4	0	1	0	0	CTC	OCR1A
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

In CTC (mode 4) mode OCIF1A can generate an interrupt when it detects a compare match. In order to activate the timer1 CTC interrupt SET(1) the OCF1A bit within the TIMSK1 register.

Fundamentos Sistemas Embarcados

```
// configura e inicia o TIMER1
void inicia_millis(unsigned long f_cpu)
{
  unsigned long ctc_match_overflow;
  ctc_match_overflow = ((f_cpu / 1000) / 8); //

  // (limpar timer qdo cooresponder a ctc_match
  TCCR1B |= (1 << WGM12) | (1 << CS11);

  // high byte first, then low byte
  OCR1AH = (ctc_match_overflow >> 8);
  OCR1AL = ctc_match_overflow;

  // habilita compare match interrupt
  TIMSK1 |= (1 << OCIE1A);

  //habilita GLOBAL INTERRUPTS
  sei();
}
```

```
int main (void)
{
  inicia_millis(16000000UL);
  pinMode(pinoLed, OUTPUT);
  Serial.begin(9600);
  while (1)
 {
    unsigned long currentMillis = nossamillis();
    Serial.println(currentMillis);
     _delay_ms(1000);
 }
}
```

Fundamentos Sistemas Embarcados

```
// rotina de interruoção do TIMER1
ISR(TIMER1_COMPA_vect)
{
  timer1_millis++;
}
```

```
int main (void)
{
  inicia_millis(16000000UL);
  pinMode(pinoLed, OUTPUT);
  Serial.begin(9600);
  while (1)
 {
    unsigned long currentMillis = nossamillis();
    Serial.println(currentMillis);
     _delay_ms(1000);
 }
}
```

```
unsigned long nossamillis ()
{
  unsigned long millis_return;
  // bloca execucao
  ATOMIC_BLOCK(ATOMIC_FORCEON) {
  millis_return = timer1_millis;
  }
  return millis_return;
}
```

Fundamentos Sistemas Embarcados

```
int main (void)
{
  inicia_millis(16000000UL);
  pinMode(pinoLed, OUTPUT);
  Serial.begin(9600);
  while (1)
 {
    unsigned long currentMillis = nossamillis();
    Serial.println(currentMillis);
     _delay_ms(1000);
 }
}
```

Monitor serial

```
51446
52475
53504
54533
55562
56591
57620
58649
59678
60706
61735
62764
63793
64822
```